VL 01 · Was ist Cloud-native AI?

AI Engineering, der AI Engineering Stack und warum Betrieb das Thema ist

Prof. Dr. Pascal Laube · HTWG Konstanz · WS 2026/27

Ein Modell, eine Frage, eine Antwort

```
$ python hallo_llm.py "Was ist die HTWG Konstanz?"

Die HTWG Konstanz ist eine private, staatlich
anerkannte Hochschule in Konstanz, ...

Modell:              meta-llama-3.1-8b-instruct
Time to First Token: 284 ms
Gesamtdauer:         664 ms
```

Frage an euch: Läuft das? Ja. Ist das ein Produkt? Was fehlt alles, damit Studierende diesem Assistenten vertrauen können?

Von 0 auf 60 ist leicht. Von 60 auf 100 nicht.

Autonomes Fahren zeigt das Muster am deutlichsten:

- Ein Shuttle, das auf einer abgesperrten Strecke bei gutem Wetter fährt, ist schnell gebaut
- Dann kommen Regen, Baustellen, Kinder auf dem Gehweg, blendende Sonne, ein Lieferwagen in zweiter Reihe
- Jeder dieser **Sonderfälle** ist selten, aber zusammen sind sie der Großteil der Arbeit

> Bei AI-Anwendungen ist es genauso. Die Demo steht nach einem Wochenende, die letzten Prozentpunkte kosten Monate.



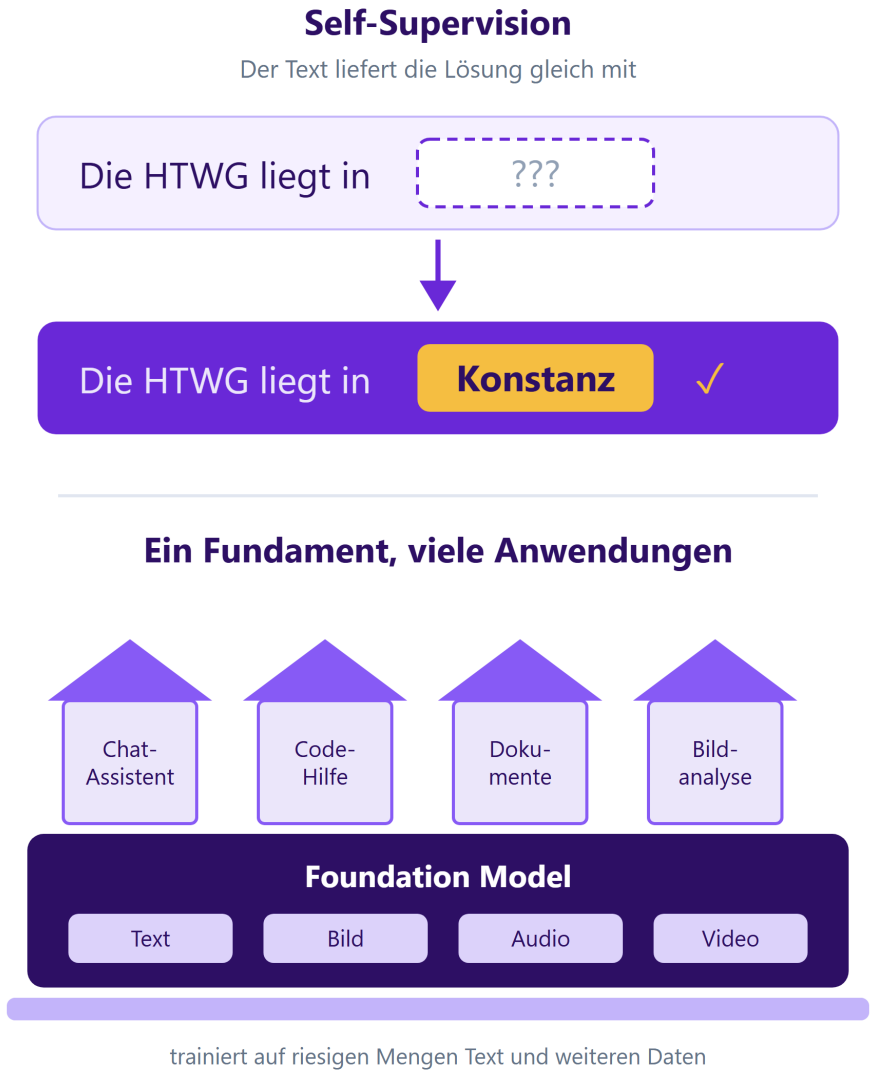
1 · Von Foundation Models zu AI Engineering

Was ist ein Foundation Model?

Self-Supervision: Ein Modell lernt, das nächste Wort eines Textes vorherzusagen. Die „Beschriftung“ steckt im Text selbst, niemand muss Daten von Hand annotieren. Deshalb kann man auf riesigen Textmengen trainieren.

Stufe	Was dazukommt
Sprachmodell	sagt das nächste Token vorher
Large Language Model	dasselbe, aber auf riesigen Datenmengen trainiert
Foundation Model	zusätzlich Bilder, Audio, Video. Ein **Fundament** für viele Anwendungen

Wie diese Modelle intern funktionieren und warum sie halluzinieren, ist Thema der nächsten Woche.



AI Engineering: Anwendungen auf Foundation Models bauen

Warum ein neuer Begriff, wenn es ML Engineering schon gibt? Drei Faktoren:

1. **Allgemeine Fähigkeiten:** Modelle lösen viel mehr Aufgaben als früher. Es gibt viel mehr sinnvolle Anwendungen.
2. **Mehr Investitionen:** AI-Anwendungen werden billiger und schneller gebaut, also lohnen sie sich öfter.
3. **Niedrige Einstiegshürde:** *Model as a Service*. Ein starkes Modell ist einen API-Aufruf entfernt, ohne eigene Infrastruktur.

AI Engineering ist nicht ML Engineering

	Klassisches ML Engineering	AI Engineering
Modell	selbst trainieren	fremdes Modell **anpassen**
Ressourcen	kleine Modelle, oft CPU	große Modelle, GPU, höhere Latenz
Ausgaben	geschlossen: Spam / kein Spam	offen: beliebiger Text, **schwer zu bewerten**

Anpassen geht auf zwei Wegen:

- **Promptbasiert:** Anweisungen und Kontext, das Modell bleibt unverändert. Schnell, wenig Daten. *Unser Weg in diesem Kurs.*
- **Finetuning:** Die Gewichte ändern sich. Aufwendiger, kann Qualität, Latenz und Kosten verbessern.

> Beispiele in den Prompt schreiben ist **kein Training**. Training heißt immer: Die Gewichte ändern sich.

2 · AI-Anwendungen planen

Bevor man baut

Welche Rolle spielt die KI?

Frage	Beispiel	Folge
Kritisch oder ergänzend?	Face ID geht nicht ohne KI, Gmail schon	je kritischer, desto zuverlässiger muss sie sein
Reaktiv oder proaktiv?	Chatbot antwortet auf Anfrage, Stauwarnung kommt ungefragt	reaktiv: schnell. Proaktiv: sehr gut, sonst nervt es
Dynamisch oder statisch?	lernt laufend aus Nutzung oder nur mit neuen Versionen	dynamisch braucht Feedback-Schleifen

Und der Mensch? *Human-in-the-Loop*: Die KI macht Vorschläge, erledigt einfache Fälle selbst oder entscheidet allein. Der schrittweise Weg dahin heißt **Crawl, Walk, Run**.

Wann ist es gut genug?

Vor dem Start braucht eine Anwendung eine **Nützlichkeitsschwelle**:

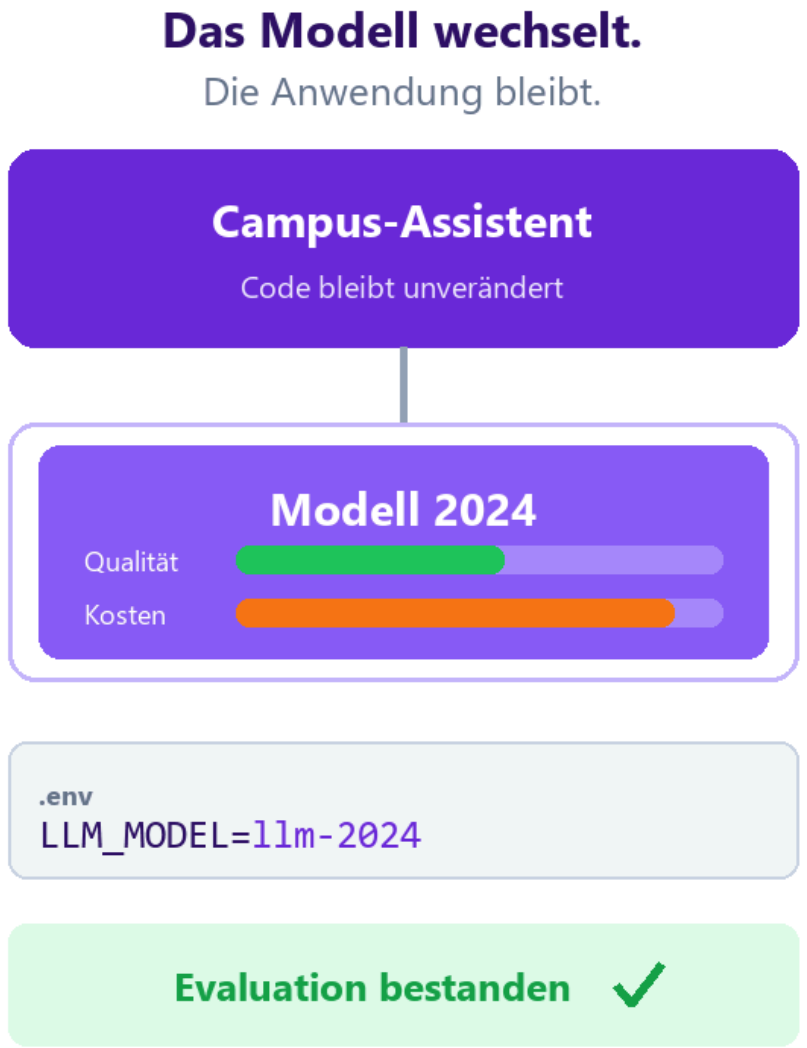
Gruppe	Kennzahlen
Qualität	Wie gut und wie korrekt sind die Antworten?
Latenz	**TTFT**: Zeit bis zum ersten Token. **TPOT**: Zeit pro weiterem Token. Gesamtdauer
Kosten	Was kostet eine Anfrage?
Weiteres	Nachvollziehbarkeit, Fairness

Qualität messen wir in VL 8, Latenz und Kosten in VL 10. Eure erste TTFT messt ihr heute in der Übung.

Eine AI-Anwendung ist nie fertig

- Modelle werden **besser und billiger**, manchmal über Nacht
- Anbieter ändern **Preise**, ändern **Modelle** oder verschwinden
- **Regulierung** kommt dazu (DSGVO, EU AI Act)
- Schnittstellen gleichen sich an, ein Modell lässt sich oft tauschen, aber **Prompts und Evaluation müssen mit**

> Konsequenz für euer System: Das Modell steht in der **Konfiguration**, nicht im Code. Und ohne Evaluation wird jeder Wechsel zum Risiko.



3 · Der AI Engineering Stack

Der AI Engineering Stack

Man startet oben und geht nur so tief wie nötig

Start

🧩 Anwendungsentwicklung

Hier entsteht heute der meiste Unterschied zwischen Produkten

- Evaluation
- Prompts und Kontext
- AI-Interface

🧠 Modellentwicklung

Klassisches ML Engineering, für Anwendungen oft nicht nötig

- Modellierung, Training
- Dataset Engineering
- Inferenzoptimierung

🏗️ Infrastruktur

Ändert sich am wenigsten, egal welches Modell gerade vorne liegt

- Model Serving
- Daten und Rechenleistung
- Monitoring

nur bei Bedarf

Wo dieser Kurs im Stack arbeitet

Schicht	Was wir in diesem Kurs damit machen
🧩 **Anwendungsentwicklung**	Hier liegt der Schwerpunkt. Ihr lernt, ein Modell mit guten **Prompts** und dem richtigen **Kontext** zu versorgen, Wissen aus Dokumenten abzurufen, **Agenten** mit Werkzeugen zu bauen und die Qualität per **Evaluation** zu messen (VL 2 bis 8)
🧠 **Modellentwicklung**	Wir trainieren keine Modelle. Wir nehmen nur die **Inferenzoptimierung** mit: Quantisierung, Batching und Caching, damit ein Modell schneller und billiger antwortet (VL 10)
🏗️ **Infrastruktur**	Der zweite Schwerpunkt. Ihr verpackt euer System in **Container**, betreibt es in **Kubernetes** mit eigenem **Model Serving**, schützt es mit einem **Gateway** und überwacht es im Betrieb (VL 9 bis 13)

AI Engineering rückt an Full-Stack heran: Früher kamen erst Daten und Modell, dann das Produkt. Heute kann man mit dem Produkt beginnen und erst in Daten und Modelle investieren, wenn sich die Idee bewährt. Genau so gehen wir vor: Heute steht ein laufender Modellaufruf, alles andere wächst Schritt für Schritt dazu.

4 · Was heißt cloud-native?

Cloud-native: Prinzipien, nicht Ort

> „Cloud native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds.“ (CNCF)

Baustein	Idee
📦 **Container**	Anwendung mit allen Abhängigkeiten verpackt, läuft überall gleich
🔁 **Unveränderliche Infrastruktur**	nicht reparieren, sondern ersetzen
📝 **Deklarative APIs**	gewünschten Zustand beschreiben, das System stellt ihn her
🧱 **Microservices**	kleine, unabhängig auslieferbare Dienste
🔭 **Observability**	Logs, Metriken, Traces

Ein Kubernetes-Cluster auf eurem Laptop ist genauso cloud-native wie einer bei einem großen Anbieter.

Cloud-native AI: dieselben Prinzipien, härtere Bedingungen

CNAI = Ansätze und Muster, um AI-Anwendungen und -Workloads nach Cloud-native-Prinzipien zu bauen und zu betreiben. Vier Dinge machen das schwerer als bei einem normalen Webservice:

⏳ Startzeit

Webservice: startet in Sekunden.
AI: Ein Modell mit vielen Gigabyte muss erst geladen werden.

🎮 Hardware

Webservice: läuft auf jeder CPU.
AI: braucht oft eine GPU, und die ist knapp und teuer.

🎲 Ausgaben

Webservice: gleiche Eingabe, gleiche Ausgabe.
AI: Antworten variieren, Qualität ist schwer zu messen.

💶 Kosten

Webservice: eine Anfrage kostet fast nichts.
AI: Jedes Token kostet, lange Antworten werden teuer.

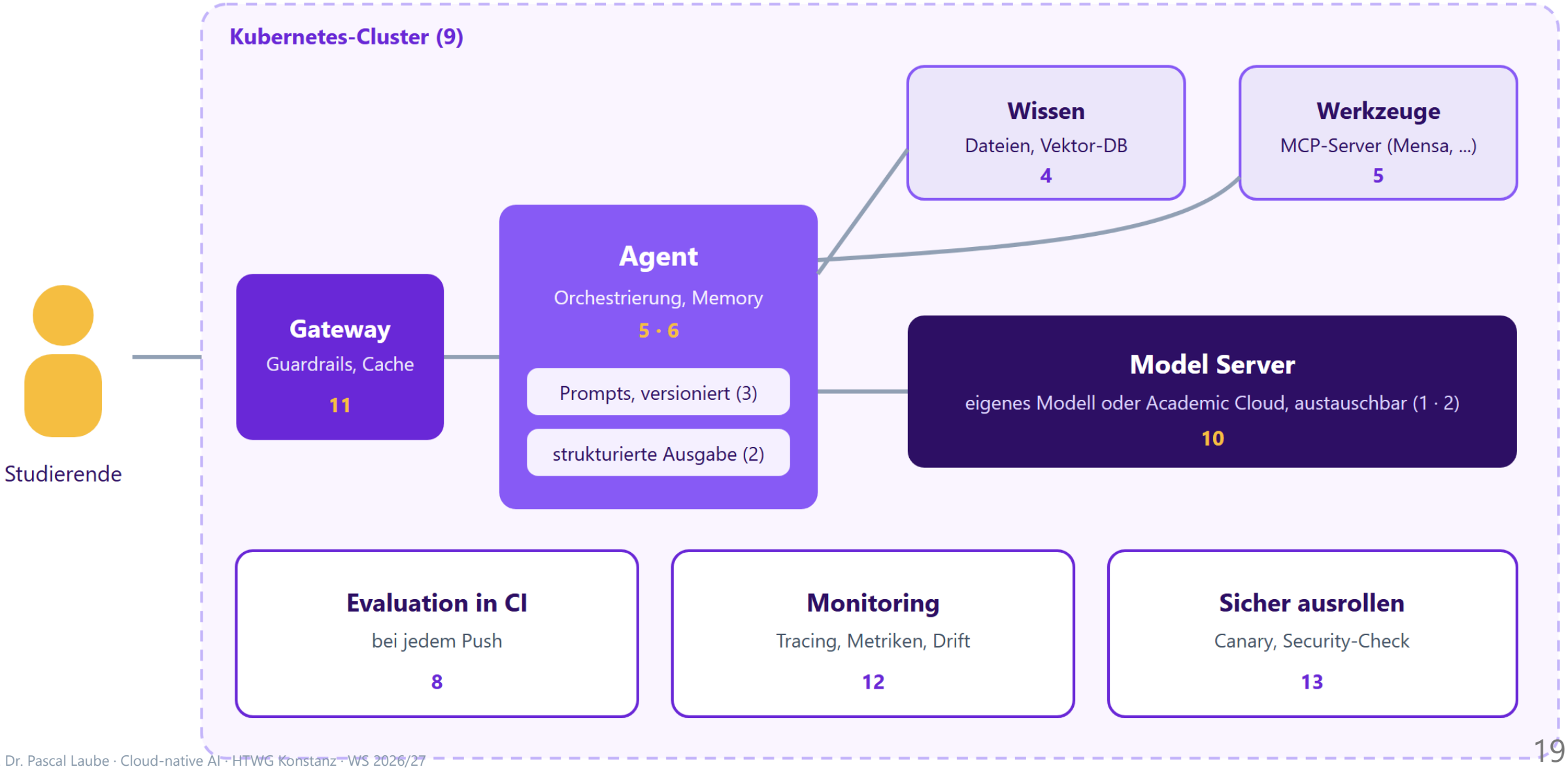
Die CNCF nennt als offene Baustellen Datenaufbereitung, Training, **Model Serving**, Nutzererfahrung und Querschnittsthemen wie Kosten und **Observability**. Wir konzentrieren uns auf Serving und Betrieb.

5 · Unser Kursbeispiel

Der Campus-Assistent

Zielarchitektur des Semesters: der Campus-Assistent

Jede Woche kommt eine Schicht dazu · die Zahlen sind die Vorlesungen



Heute: die unterste Stufe

Eine Anwendung, die ein Modell über eine **OpenAI-kompatible API** aufruft. Ollama lokal und die Academic Cloud bieten dieselbe Schnittstelle.

```
client = OpenAI(
    base_url=os.environ["LLM_BASE_URL"],
    api_key=os.environ["LLM_API_KEY"])

antwort = client.chat.completions.create(
    model=os.environ["LLM_MODEL"],
    messages=[{"role": "user",
               "content": "Was gibt es in der Mensa?"}])
```

1. `OpenAI(...)` baut den Client. *Wohin* er spricht und *mit welchem Schlüssel*, kommt aus der Umgebung.
2. `create(...)` schickt die Nachricht ans Modell und liefert die Antwort zurück.
3. `model` sagt, *welches* Modell antwortet. Auch das steht nur in `.env`.

> Gleicher Code, anderes Modell: nur `.env` ändert sich. Das Modell ist ein **austauschbarer Baustein**.

Übung ab 11:30

Teil	Aufgabe	Zeit
A	Setup: Docker, Academic-Cloud-Key beantragen, Repo anlegen	20 Min
B	Lokales Modell mit `docker compose up`, erste TTFT messen	25 Min
C	Modell tauschen, nur über `.env`. Antworten und Latenz vergleichen	25 Min
D	Reflexion im README	10 Min

> Den Academic-Cloud-Key **gleich am Anfang** beantragen, die Freischaltung kann dauern. Und `.env` kommt nie ins Repository.

Quellen

- Chip Huyen: *AI Engineering. Building Applications with Foundation Models*. O'Reilly, 2025, Kapitel 1
- CNCF AI Working Group: *Cloud Native Artificial Intelligence*. Whitepaper, 2024
- Cloud Native Computing Foundation: *CNCF Cloud Native Definition v1.1*
- Microsoft: *Crawl, Walk, Run*, schrittweise Automatisierung mit KI (zitiert nach Huyen, Kap. 1)
- Apple: *Human Interface Guidelines, Machine Learning* (zitiert nach Huyen, Kap. 1)

Eine Demo ist schnell gebaut. Ein System muss man betreiben.

Nächste Woche: Wie funktionieren Foundation Models, und warum erfinden sie Dinge?